

test

August 16, 2025

```
[17]: import random
import torch
```

```
[18]: def synthetic_data(w, b, num_examples):
    x = torch.normal(0, 1, (num_examples, len(w)))
    y = torch.matmul(x, w) + b
    y += torch.normal(0, 0.01, y.shape)
    return x, y.reshape((-1, 1))

true_w = torch.tensor([2, -3.4])
true_b = 4.2
features, labels = synthetic_data(true_w, true_b, 1000)
```

```
[19]: def data_iter(batch_size, features, labels):
    num_examples = len(features)
    indices = list(range(num_examples))
    random.shuffle(indices)
    for i in range(0, num_examples, batch_size):
        batch_indices = torch.tensor(
            indices[i: min(i + batch_size, num_examples)]
        )
        yield features[batch_indices], labels[batch_indices]

batch_size = 10

for x, y in data_iter(batch_size, features, labels):
    print(x, "\n", y)
    break
```

```
tensor([[ 1.2798, -0.5897],
        [-0.4909, -0.8965],
        [-0.3424,  0.3196],
        [-1.3090,  1.7403],
        [-0.1421, -0.4544],
        [ 0.0272, -0.8076],
        [ 1.3122, -0.4370],
```

```

        [-0.9411, -0.5767],
        [-0.8651,  0.5981],
        [-0.7637, -0.9613]])
tensor([[ 8.7614],
        [ 6.2801],
        [ 2.4084],
        [-4.3412],
        [ 5.4533],
        [ 7.0035],
        [ 8.3094],
        [ 4.2876],
        [ 0.4424],
        [ 5.9447]])

```

```

[20]: w = torch.normal(0, 0.01, size=(2, 1), requires_grad=True)
      b = torch.zeros(1, requires_grad=True)

```

```

[21]: def linreg(x, w, b):
      return torch.matmul(x, w) + b

      def squared_loss(y_hat, y):
          return (y_hat - y.reshape(y_hat.shape)) ** 2 / 2

      def sgd(params, lr, batch_size):
          with torch.no_grad():
              for param in params:
                  param -= lr * param.grad / batch_size
                  param.grad.zero_()

```

```

[22]: lr = 0.03
      num_epochs = 10
      net = linreg
      loss = squared_loss

      for epoch in range(num_epochs):
          for x, y in data_iter(batch_size, features, labels):
              l = loss(net(x, w, b), y)
              l.sum().backward()
              sgd([w, b], lr, batch_size)
          with torch.no_grad():
              train_l = loss(net(features, w, b), labels)
              print(f"epoch: {epoch + 1}, loss: {float(train_l.mean())}")

```

```

epoch: 1, loss: 0.031901322305202484
epoch: 2, loss: 0.00011468322918517515
epoch: 3, loss: 5.235545540926978e-05
epoch: 4, loss: 5.2269326261011884e-05
epoch: 5, loss: 5.2335071814013645e-05

```

epoch: 6, loss: 5.243266059551388e-05
epoch: 7, loss: 5.2304087148513645e-05
epoch: 8, loss: 5.2301958930911496e-05
epoch: 9, loss: 5.231380055192858e-05
epoch: 10, loss: 5.2282648539403453e-05